Key Idea

Key Idea

The key idea of hashing is using a function h to map a large universe U to a small range $\{0, 1, 2, \cdots, m-1\}$. Then we can use an array A of size m to store. $A[i]$ is called a **slot**.

The function h is called **hash function**.

We should try to ensure that for $x \neq jy$, $h(x) \neq h(y)$. However, if $|U| > m$, there must exist $x \neq y$ such that $h(x) = h(y)$, which is called **collision**.

We should find some good hash function to reduce collisions. And we should use some technics to ensure that if collision happens, we can also get the correct answer.

Load factor is defined as

$$\alpha = \frac{\text{number of elements}}{\text{number of slots}}.$$

Direct Addressing Table

Direct Addressing Table

Given $U = \{0, 1, 2, \cdots, m - 1\}$, we can use the hash function

$$h(x) = x.$$

Then for $x \neq y$, we must have $h(x) \neq h(y)$. There are no collisions. In each slot, we just store the element itself.

The number of elements cannot exceed the number of slots, so the load factor $\alpha \leq 1$.

Direct Addressing Table

Limitation: If the universe of keys is usually very large, the space cost will be very large.

Hash Function

Hash Function

When U is small, we can use an injection to map U to $\{0, 1, 2, \cdots, m-1\}$. Then there are no collisions. In each slot, we can also store the element itself.

Given $U = \{a, a+1, a+2, \cdots, b\}$ and $b - a$ is small, we can use the hash function

$$h(x) = x - a.$$

Hash Function

Sometimes, U is not a set of integer.

Given U = {string of length 3} and the charset is ASCII, we can use the hash function

$$h(s) = s[0] + s[1] \times 128 + s[2] \times 128^2.$$

It means s can be treated as an integer of base 128.

Exercise

Given U = {string of length ≤ 3} and the charset is ASCII, what hash function can be used?

Division Hashing

When U is large, for example, $U = \{0, 1, 2, \cdots, 2^{32} - 1\}$, we can not use an injection because it will make m large.

If U is a set of integer, we can use **division hashing**. The hash function is defined as

$$h(x) = x \bmod m.$$

Then we map U to $\{0, 1, 2, \cdots, m - 1\}$.

Division Hashing

$$h(x) = x \bmod m$$

How to select m?

If the input data are generated randomly, we can use any m. In practice, input data are from real life and the data may be regular.

For example, if you want to store scores in exams, the data may be all multiples of 2. If we use an even number as m, the probability of a collision will increase.

So we need to choose a large prime number as m.

Collision resolution

Chaining

When collision happens, we store multiple elements in one slot.

So we use array of linked lists instead arrays of elements, i.e, there is a linked list in each slot.

When we add an element x into the hash table, we place x into the linked list of the slot $h(x)$.
When we delete an element x from the hash table, we remove x from the linked list of the slot $h(x)$.
When we find an element x in the hash table, we find x in the linked list of the slot $h(x)$.

Since there may be multiple elements in one slot, the load factor α may be greater than 1.

Open-addressing

When collision happens, we find another slot to store it.

We extend the hash function as

$$h'(x, i) : U \times \{0, 1, \cdots, m - 1\} \to \{0, 1, \cdots, m - 1\}.$$

When we add an element x into the hash table, we get a **probe sequence**

$$[h'(x, 0), h'(x, 1), h'(x, 2), \cdots, h'(x, m - 1)].$$

Each element in the probe sequence represents a slot. We sequentially check whether the slot is empty. If so, we place x in that slot. Otherwise, we continue checking.

Open-addressing

OPEN-ADDRESSING-HASH-TABLE-ADD $(x, value)$

1 **for** $pos \in [h'(x,0), h'(x,1), h'(x,2), \cdots, h'(x,m-1)]$
2 **if** $A[pos] =$ NIL
3 $A[pos] = x \mapsto value$

Open-addressing

When we find an element x in the hash table, we sequentially check whether the slot in probe sequence is empty. If so, x is not in the hash table. Otherwise, we check whether the element in the slot equals to x. If so, x is in the hash table. Otherwise, we continue checking.

OPEN-ADDRESSING-HASH-TABLE-FIND (x)

```
1  for pos ∈ [h'(x, 0), h'(x, 1), h'(x, 2), ⋯, h'(x, m − 1)]
2      if A[pos] = NIL
3          return NIL
4      elseif A[pos].key = x
5          return A[pos].value
```

Open-addressing

How to delete?

We cannot mark the deleted one as empty.

Assume that there exist $x_0 \neq x_1 \neq x_2$ such that the probe sequence of x_0, x_1, x_2 are equal. We add x_0, x_1, x_2 to the hash table sequentially.

x_0 in the slot of $h'(x_0, 0)$.
x_1 in the slot of $h'(x_0, 1)$.
x_2 in the slot of $h'(x_0, 2)$.

If we delete x_1 from the hash table, x_2 can not be found. (After finding that $A[h'(x_2, 1)]$ is empty, we will not look for $A[h'(x_2, 2)]$ anymore.)

Open-addressing

We should mark the deletion differently.

OPEN-ADDRESSING-HASH-TABLE-2-DELETE (x)

```
1  for pos ∈ [h′(x, 0), h′(x, 1), h′(x, 2), ⋯, h′(x, m − 1)]
2      if A[pos] = NIL
3          return
4      elseif A[pos] = DELETED
5          continue
6      elseif A[pos].key = x
7          return A[pos] = DELETED
```

Open-addressing

Adding and finding should also be modified.

OPEN-ADDRESSING-HASH-TABLE-2-ADD (x)

1 **for** $pos \in [h'(x,0), h'(x,1), h'(x,2), \cdots, h'(x,m-1)]$
2 **if** $A[pos]$ = NIL or $A[pos]$ = DELETED
3 $A[pos] = x \mapsto value$

Open-addressing

```
OPEN-ADDRESSING-HASH-TABLE-2-FIND(x)
1  for pos ∈ [h'(x,0), h'(x,1), h'(x,2), ⋯, h'(x,m − 1)]
2      if A[pos] = NIL
3          return
4      elseif A[pos] = DELETED
5          continue
6      elseif A[pos].key = x
7          return A[pos].value
```

Open-addressing

Since there is at most one element in one slot, the load factor $\alpha \leq 1$.

Open-addressing

What happens if the number of elements to be added exceeds m?

When the load factor α is greater than a threshold (0.75 in practice), we choose a larger m' ($2m$ in practice) to construct a new hash table. Then the elements in the old hash table are added into the new hash table one by one.

Open-addressing

How to select h'?

Linear Probing:

$$h'(x, i) = (h(x) + i) \bmod m.$$

Double hashing:

$$h'(x, i) = \left(h_1(x) + i h_2(x)\right) \bmod m.$$

Note that $\gcd\left(h_2(x), m\right) = 1$ for all x.